Exercise 12

Decoding Chessboard Configurations

In this exercise, you get to use binary file I/O and bitwise operators to help with a simple compression task. You'll be reading input that represents a collection of chess pieces on a chess board. You'll use this to fill in a 2D array representing the chessboard and print it out. In the input, the description of each piece is encoded so that the piece's color, its type, its row position and its column position are stored in different bit fields of a single, sixteen-bit unsigned short. You'll need to use binary file IO to read the short values from an input file, and you'll need to use the bitwise operators to extract the values for these fields so that you can fill in the chessboard as a 2D array.

On the course homepage, you'll find a source file, **chessboard.c** with input and expected output files. The input files are in binary, so if you download them via the webpage, do this by right-clicking on each input file and choosing to download it. Don't try to open the file in your browser and then copy-and-paste it into a text editor. Alternatively, if you're logged in on remote.eos.ncsu.edu, you can copy these files to your current directory with the following shell command:

```
cp /mnt/coe/workspace/csc/CSC230/ex/exercise12/* .
```

To complete the working program, you'll need to fill in code for the following functions:

- `void decodePiece( unsigned short input, char board[SIZE][SIZE] )`
  This function takes a chess piece's encoded information as input. Based on this, it fills in the appropriate cell in the given board to represent the piece. Below, we talk about how a piece is encoded as a 16-bit input value.
- `void printText( char board[SIZE][SIZE] )`
  This function prints the chessboard in a textual form with ASCII characters. The light pieces are printed with capital letters and the dark ones are printed with lower-case. In the Output section below, we'll talk more about the output specification.

In `main`, you need to add code to read a sequence of unsigned 16-bit values from a file given on the command line. The input file could contain any number of 16-bit values, so you will need to read up to the end-of-file. Also, the values are stored in binary (rather than text), so you will need to make sure you open the input file in binary mode and read values with fread() to copy them directly from the file into memory. For each value, you will call decodePiece() to fill in the appropriate board location based on the piece encoding.

If you can't open the input file given on the command line, you program should print the following message to standard error and then terminate with an exit status of 1. For *filename*, report the name of the file given on the command line.

`Can't open file: `*`filename`*

Chessboard Representation

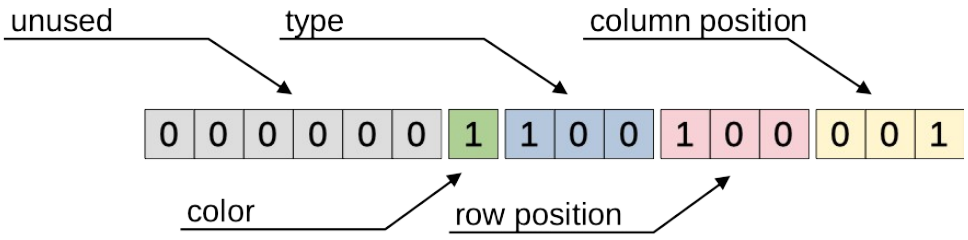
The chessboard is represented as a 2D array of char values, with each element representing what's at a particular board location. The rows of this array are not null terminated strings. They're just arrays containing 8 char values for the 8 pieces on a row of the chess board.

In the chessboard, we use a value of zero to represent an empty space. For non-empty spaces, we use an ASCII character code indicating the type of piece in that location. The following table shows what characters are used to represent each piece. For example, the letter 'R' represents a light-side rook, and the letter 'n' represents a dark-side knight.

Chess Piece	Dark Character	Light Character
Pawn	p	P
Knight	n	N
Bishop	b	B
Rook	r	R
Queen	q	Q
King	k	K

Chess Piece Encoding

The input will consist of a sequence of 16-bit values in binary. The following figure shows how information about a piece is stored in a 16-bit value.



The low-order (least-significant) three bits give the column number for the piece, starting from column zero on the left up to column 7 on the right. The next three bits give the row number, from row zero at the top to row 7 at the bottom. The next three bits indicate the type of piece. The following table says how to interpret this value. There are also preprocessor constants in the starter code for each type of piece.

Type Value	Chess Piece
1	Pawn
2	Knight
3	Bishop
4	Rook
5	Queen
6	King

The next bit (bit number 9) indicates whether the chess piece is light (a 1 bit) or dark (a 0 bit). The remaining six high-order bits are unused, but they may not always be zero. You will need to ignore these bits when you're decoding a piece from the input.

The example piece in the figure above describes a light-side Rook in row 4, column 1. This is the first sample input value in the input-1.bin file provided with this exercise.

The input values will all be valid. They may have arbitrary values in the high-order six bits, but the bit field for the type of piece will always contain a number between 1 and 6 and two or more pieces will never be assigned to the same chessboard location.

Your decodePiece() function should extract the fields for a given piece and fill in the corresponding element of the board array. For each piece, the location in the chessboard will be determined by the row and column values in the input. The particular character value for that piece depends on the type of piece and its color.

ASCII Chess Board Output

In your printText() function, you'll need to output the chessboard as ASCII characters. Each row of the board will be printed as a row of text, with row zero at the top and row 7 on the bottom. Each row will have a column for each column of the board (column zero on the left) and a blank space between columns. Each piece will be printed as its capital or lower-case letter, as shown in the table above. Empty spaces should be printed as a dash character, '-'.

Graphical Chess Board Output

The starter for chessboard.c contains some code to print out the board using symbols (rather than letters) for the chess pieces. You won't need to change this code, but you should be able to use it if you want, to get a better-looking picture of the chessboard. You just need to give a "-g" option on the command line, before the name of the input file.

Sample Execution

You should be able to compile your program with the following gcc command.

```
$ gcc -Wall -std=c99 chessboard.c -o chessboard
```

Once your program is ready, you should be able to run it as follows for input-1.bin.

```
$ ./chessboard input-1.bin
r - - - - - - -
r p k - - - - -
- - - - - - - -
- - - - - - - -
P R K - - - - -
R - - - - - - -
- - - - - - - -
- - - - - - - -
```

To make sure your program's output is exactly right, you can save it to a file then compare that to the expected output.

```
$ ./chessboard input-1.bin > output.txt
$ diff output.txt expected-1.txt
```

The first sample input just contains a few chess pieces, some light and some dark. The second input includes more pieces and some non-zero bits in the high-order six bits of the input values. Your program should be ignoring these bits. This input file should help you to check that this part of your code is working.

```
$ ./chessboard input-2.bin
- Q - - - - - -
- - - - - p k -
- - p - - - p -
- p - - N - - p
- b - - - - - P
- b n - - - - -
- - r - - - P -
- - K - - - - -
```

Enabling Graphical Output

It turns out the unicode character set already includes characters for standard chess pieces. The starter code has a function to print the board using these special, non-ASCII characters and code in main() to decide which printing function to use based on the command-line arguments. Once the rest of your program is working, you should be able to run it as follows, to see the board drawn with these special-purpose unicode characters. We tried this on several of the terminal programs you might be using, and it seemed to work on all of them. You might need to increase your font size a little bit to see the chess pieces better.

```
$ ./chessboard -g input-1.bin
♜ . . . . . . .
♜ ♟ ♚ . . . . .
. . . . . . . .
. . . . . . . .
♙ ♖ ♔ . . . . .
♖ . . . . . . .
. . . . . . . .
. . . . . . . .
```

Submitting your Solution

When you're done, submit the source for your completed chessboard.c to the exercise_12 assignment on Moodle.